

Chapter 1

Temporal-Difference Learning

Contents

1	Introduction	2
2	TD Prediction	2
2.1	Algorithm	2
2.1.1	Convergence of the Algorithm	3
2.2	TD Error	3
2.3	Batch Updates on TD	3
3	TD Control	4
3.1	Sarsa: On-policy TD Control	4
3.2	Q-learning: Off-policy TD Control	4

1 Introduction

Temporal-Difference learning (TD) is one of the central ideas in reinforcement learning. Like DP, TD methods update estimates based in part on other learned estimates. On the other hand, TD methods learn directly from experiences like MC methods. DP, MC, and TD methods' relationships will be kept founded in later chapters. n-step algorithms and TD(λ) algorithms are examples of them.

The main advantages of TD over MC and DP methods is as following:

- Do not have to wait until the epsidoes ends.
- Do not have to ignore or discount episodes on which experimental actions are taken.
- No model is required.

2 TD Prediction

TD method's advantage over MC method is that it can be applied to continuing tasks, since it updates each time step. The simplest form of TD method makes the update

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)] \quad (1.1)$$

immediatly on transition to S_t receiving R_{t+1} . This method is called **TD(0)**, and is a speacial case of n-step bootstrap and TD(λ).

2.1 Algorithm

Algorithm 1: Tabular TD(0) Prediction

Input : π policy to be evaluated
 $\alpha \in (0, 1]$ step size parameter
Output: v_π state-value function of policy π

// initialize

1 **foreach** $s \in \mathcal{S}$ **do**

2 $V(s) \leftarrow \begin{cases} \text{arbitrary value} & \text{if } s \in \mathcal{S}^+ \\ 0 & \text{if } s \in \mathcal{S} \setminus \mathcal{S}^+ \end{cases}$

3 **foreach** episodes **do**

4 $S \leftarrow \text{arbitrary } s \in \mathcal{S}^+$

5 **foreach** step in episode **do**

6 $A \leftarrow \text{action given by } \pi \text{ for } S$

7 take action A , receive reward R and next state S'

8 $V(S) \leftarrow V(S) + \alpha [R + \gamma V(S') - V(S)]$

9 $S \leftarrow S'$

10 **if** $S \in \mathcal{S} \setminus \mathcal{S}^+$ **then**

11 break

12 **return** $V \simeq v_\pi$

Since TD(0) method updates value with existing estimates, we say that it **bootstraps**, like DP method. TD combines the sampling of MC and bootstrapping of DP, and we call this update as **sampling update**. Note that it is different from *expected update* of DP.

2.1.1 Convergence of the Algorithm

Since TD(0) method uses estimates instead of complete G_t , one may be skeptical of its convergence. However, it is known to converge to v_π , happily. For any fixed policy π , TD(0) is known to converge to v_π with constant step-size parameter if it is sufficiently small, and is known to converge to v_π with probability 1 if the step-size parameter changes step by step while satisfying the stochastic approximation conditions ??.

There is no known answer to the question "If TD and MC both converges to correct answer, then which one converges more quickly?". In practice, TD methods usually converge faster than MC methods on stochastic tasks.

2.2 TD Error

The following definition of *TD error* arises in various forms throughout reinforcement learning:

Definition 2.2.1 (TD Error)

Let V be the estimate of state-value function. Then, the **TD error** δ_t at time t is defined as

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \quad (1.2)$$

Note that TD error is also contained in algorithm 1. If the estimate V does not change during the episode, the Monte Carlo error $G_t - V(S_t)$ can be expressed as the weighted sum of TD errors;

$$G_t - V(S_t) = \sum_{k=t}^{T-1} \gamma^{k-t} \delta_t. \quad (1.3)$$

Even though V changes during episode, the equation above may still hold approximately if the step size is small enough. Generalization of this idea plays an important role in the theory and algorithms of TD learning.

2.3 Batch Updates on TD

We can apply batch updates to TD(0) methods by computing the increments 1.1 every time step but updating the value after the batch, by the sum of all the increments.

Very interesting property can be found on batch TD(0) prediction. Batch TD(0) prediction converges to the **certainty-equivalence estimate** of the MDP. Certainty-equivalence estimate of MDP assumes that the estimate of the MDP is perfect. That is, it is *certain* about its estimate.

To calculate certainty-equivalence estimate directly, it requires memory complexity of $\mathcal{O}(n^2)$ due to the transition probability table, and time complexity of $\mathcal{O}(n^3)$ due to solving the Bellman equation as the linear system of equations. However, using TD(0) method, it requires memory complexity of only $\mathcal{O}(n)$, to save each values of states, which is a surprising result.

Remark 2.3.1 (TD(0) Converges to Certainty-equivalence Estimate)

uppose that the algorithm converged to fixed point. Then, the next batch update of value of each state s should be 0.

$$\sum_{k \in \text{batch}} \sum_{t \in \{t: S_t^k = s\}} \alpha(R_{t+1}^k + \gamma V(S_{t+1}^k) - V(s)) = 0 \quad (1.4)$$

Let $N(s)$ and $N(s, s')$ denote the number of visits of state s in batch and the number of transition from s to s' in batch, respectively. Then, the equation 1.4 can be written as

$$\left(\sum_{t \in \{t: S_t = s\}} R_{t+1} \right) + \gamma \left(\sum_{s'} N(s, s') V(s') \right) - N(s) V(s) = 0. \quad (1.5)$$

Dividing both side of equation 1.5 with $N(s)$, we get

$$\frac{1}{N(s)} \left(\sum_{t \in \{t: S_t = s\}} R_{t+1} \right) + \gamma \sum_{s'} \frac{N(s, s')}{N(s)} V(s') - V(s) = 0 \quad (1.6)$$

Then, the term $\hat{R}(s) = \frac{1}{N(s)} \left(\sum_{t \in \{t: S_t = s\}} R_{t+1} \right)$ is the average reward of state s and the term $\hat{P}(s'|s) = \frac{N(s, s')}{N(s)}$ is the MLE estimate of transition probability from s to s' . Substituting, we get

$$V(s) = \hat{R}(s) + \gamma \sum_{s'} \hat{P}(s'|s) V(s'), \quad (1.7)$$

which equals the Bellman equation with certainty-equivalence estimate.

3 TD Control

3.1 Sarsa: On-policy TD Control

3.2 Q-learning: Off-policy TD Control